

Image Recognition

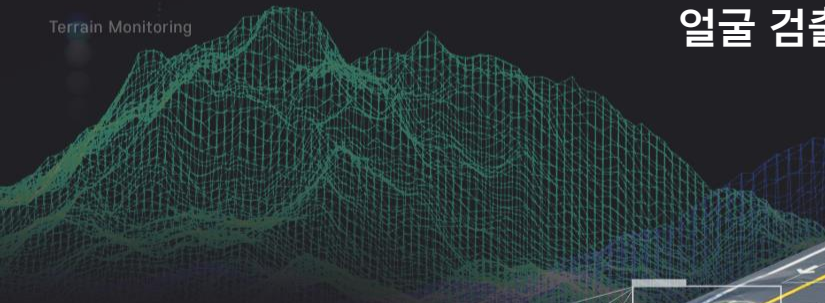


강원혁신플랫폼

컴퓨터 비전

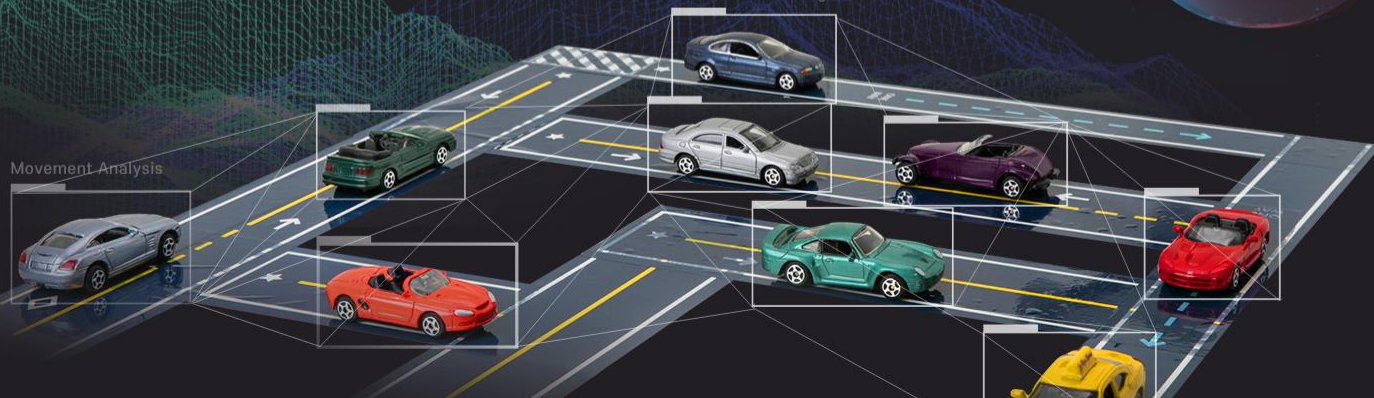
얼굴 검출(haar cascade) 이미지

Terrain Monitoring



Autonomous Driving

Movement Analysis



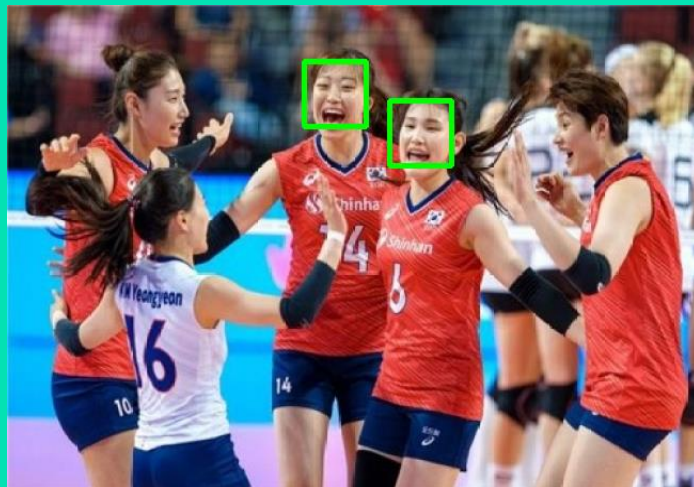
■ 이미지에서 얼굴 검출(Image Face Detection)

- Haar Cascade(하르 특징 분류기) 객체 감지 방식은 사전에 학습된 haarcascade module을 사용하여 이미지에서 얼굴과 눈 등을 감지함

이미지에서 정면 얼굴 검출한 결과

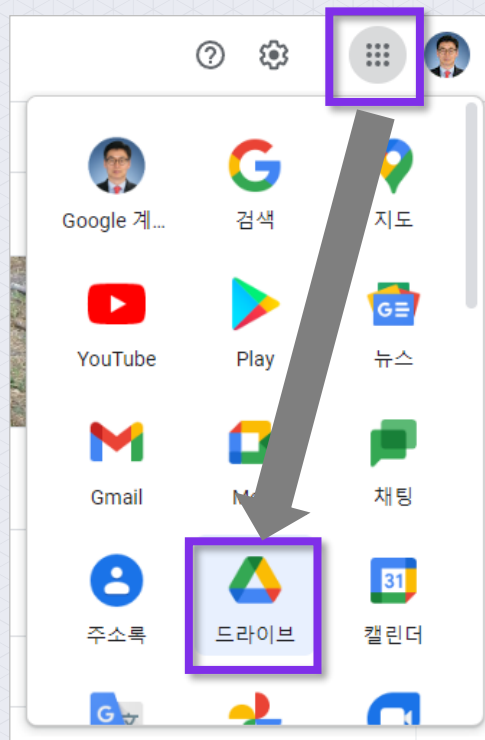


원본 이미지



정면 얼굴 검출 이미지

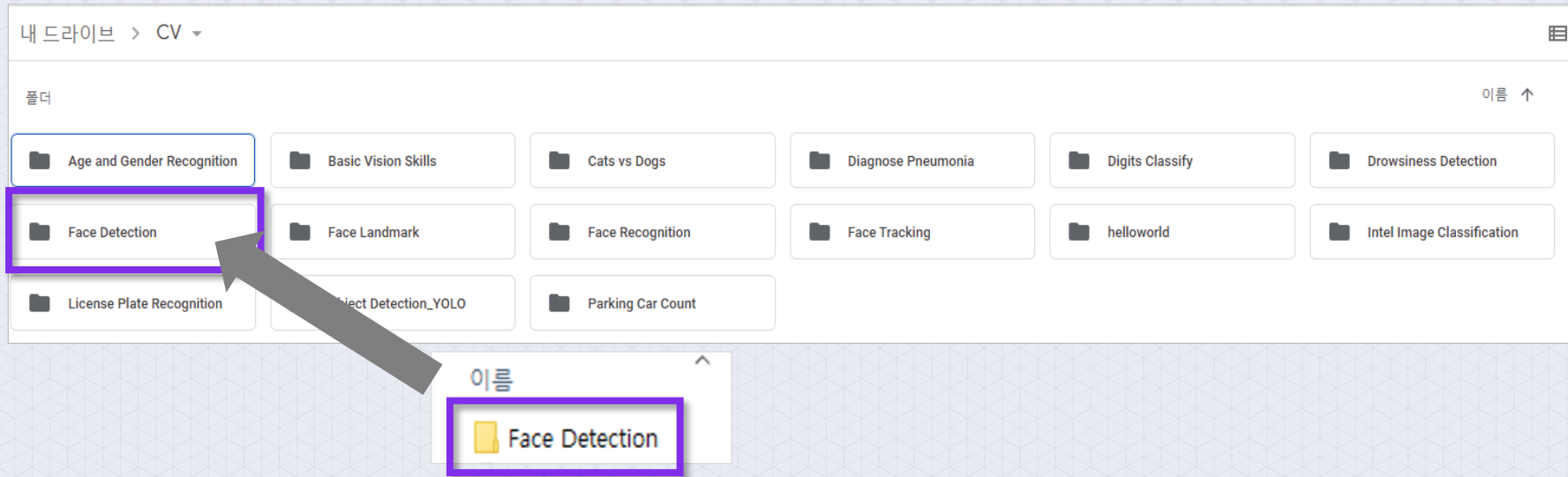
- 먼저, 구글 Colab을 이용해 보자
- 구글 크롬 브라우저에서 아래 그림과 같이 **구글 드라이브**를 클릭



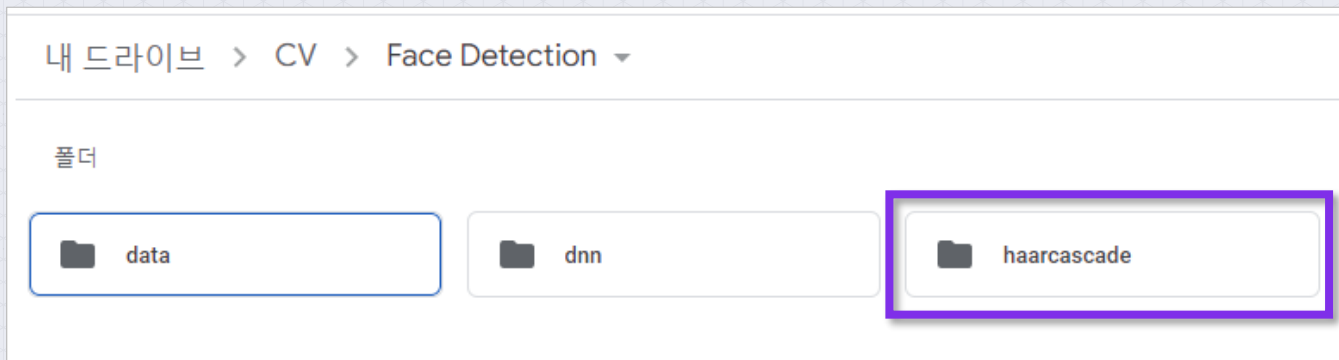
구글 드라이브에서 CV 폴더를 클릭

The screenshot shows the Google Drive web interface. On the left sidebar, the '내 드라이브' (My Drive) section is active. Below it, the '폴더' (Folders) section displays a list of folders: 'ipynb_checkpoints', '구글문서' (Google Docs), '구글시트' (Google Sheets), '구글피티' (Google Slides), '스프레드시트' (Spreadsheets), and 'Colab Notebooks'. The 'CV' folder is highlighted with a purple rectangular box. The main content area shows a '추천' (Recommended) section with various image files, including 'Cats vs Dogs.ipynb' and several 'dog.***.jpg' files. The top navigation bar includes the Google Drive logo, a search bar, and user profile icons.

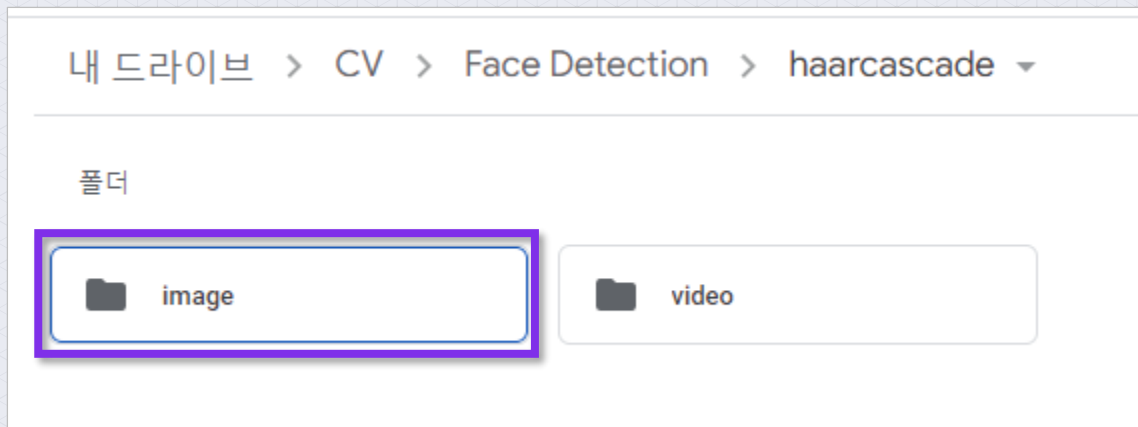
- 구글 드라이브 > CV > Face Detection 폴더를 클릭
- 강의 실습 자료를 다운로드하여 [Face Detection] 폴더를 구글 드라이브 CV 폴더 아래에 복사(업로드)



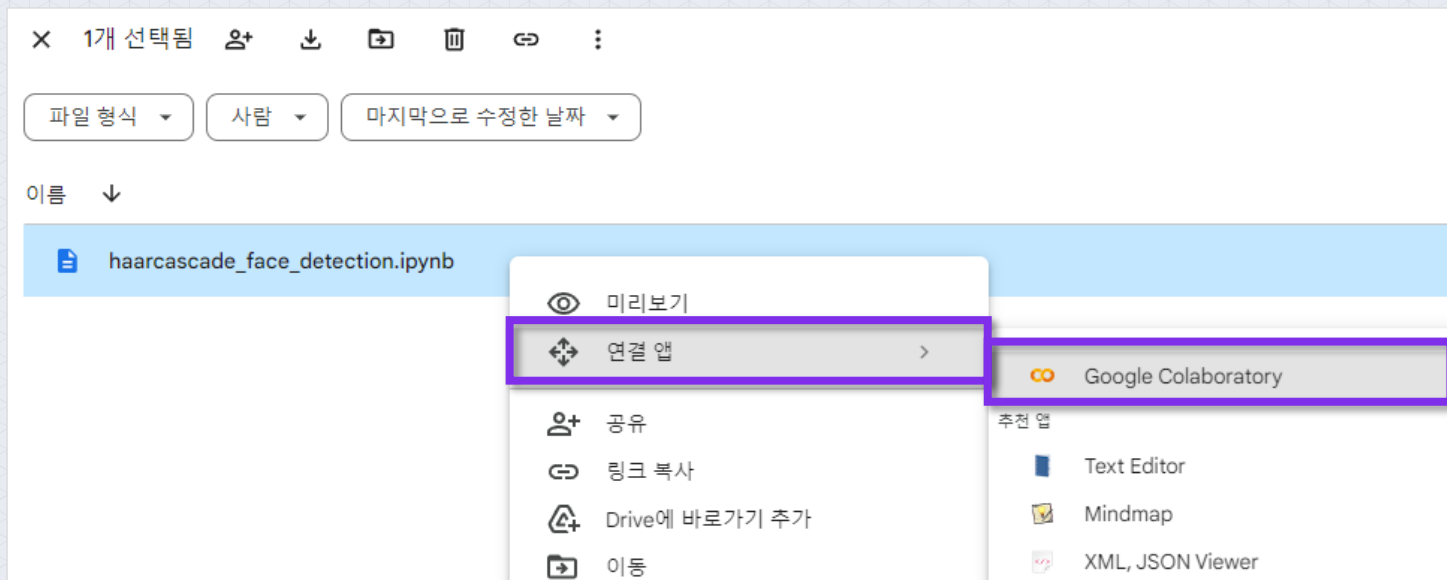
- ➡ 구글 드라이브 > CV > Face Detection > haarcascade 폴더 클릭
- ➡ Face Detection > haarcascade 폴더 클릭



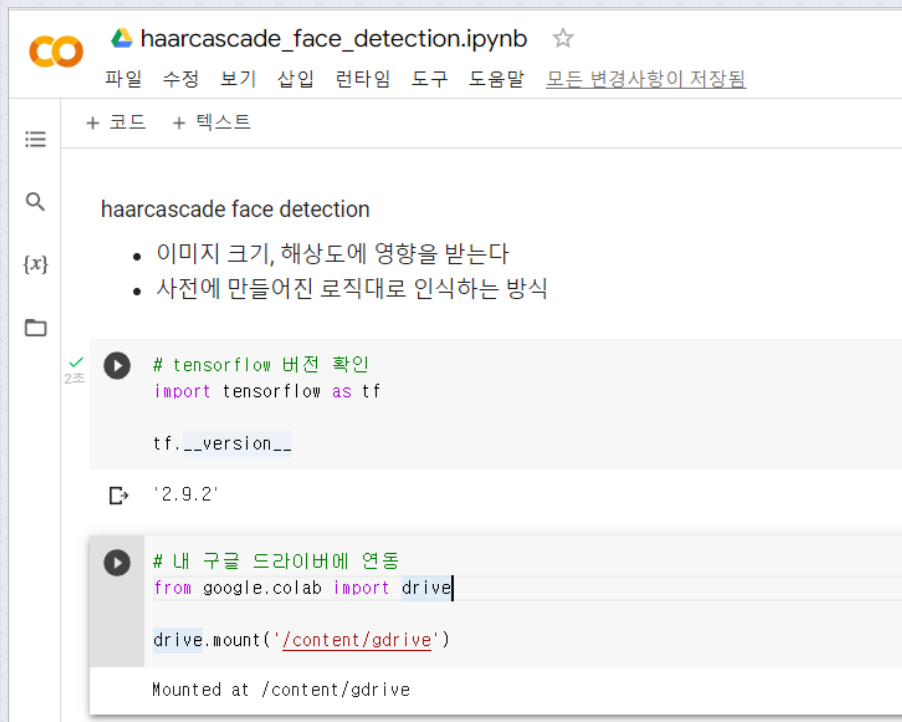
- ➡ 구글 드라이브 > CV > Face Detection > haarcascade > image 폴더 클릭
- ➡ Face Detection > haarcascade > image 폴더 클릭



- 🔄 image 폴더에서 `haarcascade_face_detection.ipynb`를 선택 후 오른쪽 마우스를 클릭하여 “연결 앱 > Google Colaboratory” 클릭



구글 Colab 노트북 화면이 보이면 실습 준비가 완료된 것



haarcascade_face_detection.ipynb ☆

파일 수정 보기 삽입 런타임 도구 도움말 모든 변경사항이 저장됨

+ 코드 + 텍스트

haarcascade face detection

- 이미지 크기, 해상도에 영향을 받는다
- 사전에 만들어진 로직대로 인식하는 방식

```
# tensorflow 버전 확인
import tensorflow as tf

tf.__version__
```

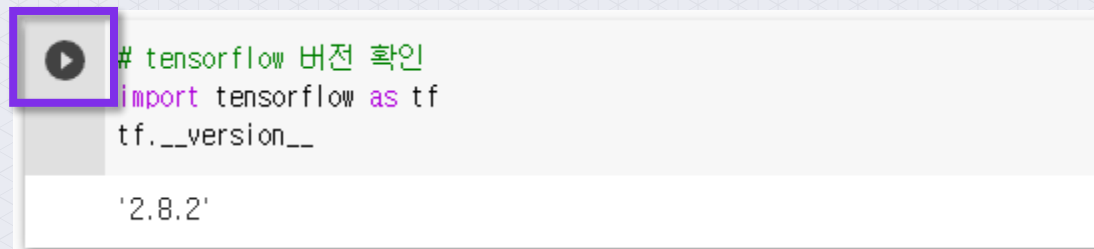
'2.9.2'

```
# 내 구글 드라이브에 연동
from google.colab import drive

drive.mount('/content/gdrive')
```

Mounted at /content/gdrive

- 구글 Colab 노트북 화면에서 실행 버튼을 클릭



```
# tensorflow 버전 확인
import tensorflow as tf
tf.__version__

'2.8.2'
```

- 실행 결과 tensorflow 버전이 '2.8.2'인 것을 알 수 있음

- ② 내 구글 드라이브에 연동
- ② 구글 Colab 노트북 화면에서 실행 버튼 클릭



- ② [Google Drive에 연결] > [계정 선택] > [구글 드라이브 접근 허용] 메뉴를 선택하면 gdrive에 연동
- ② gdrive에 연동되면 구글 드라이브에 저장된 데이터를 활용할 수 있음

구글 Colab 화면에서 내 구글 드라이브 연동

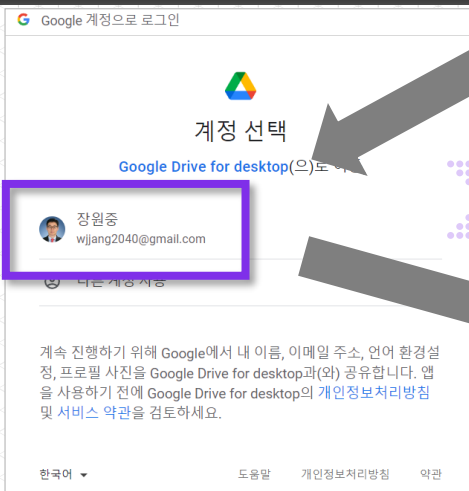
- [Google Drive에 연결] > [계정 선택] > [구글 드라이버 접근 허용]
- 아래 화면을 참고하여 **gdrive**에 연동

노트북에서 Google Drive 파일에 액세스하도록 허용하시겠습니까?

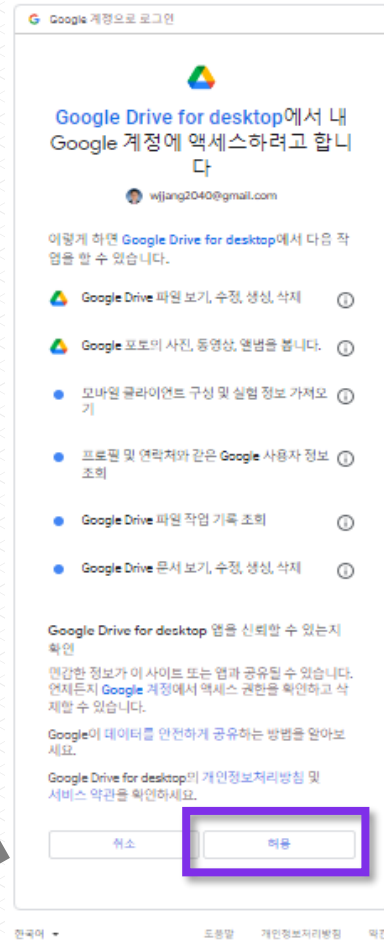
이 노트북에서 Google Drive 파일에 대한 액세스를 요청합니다. Google Drive에 대한 액세스 권한을 부여하면 노트북에서 실행되는 코드가 Google Drive의 파일을 수정할 수 있게 됩니다. 이 액세스를 허용하기 전에 노트북 코드를 검토하시기 바랍니다.

아니오

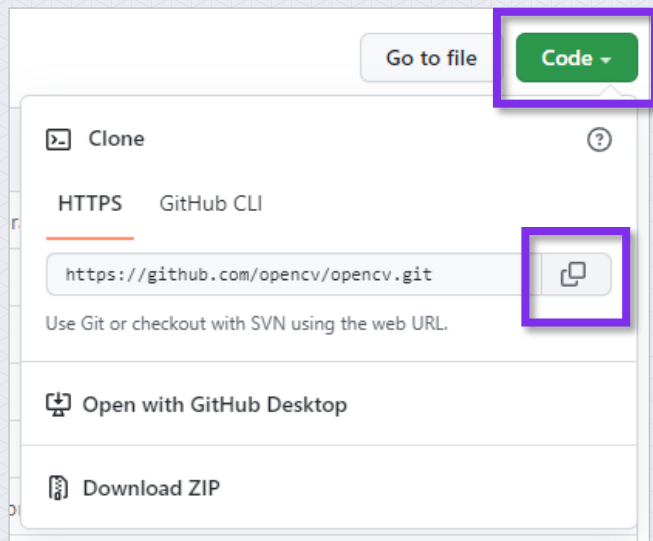
Google Drive에 연결



구글 드라이버 접근 허용



- ☞ 사전에 학습된 haarcascade module이 GitHub에 있으므로 Colab에서 GitHub와 연동하여 dataset을 다운로드 받음
- ☞ <https://github.com/opencv/opencv>에 접속
- ☞ [Code] 버튼을 클릭하면 다운로드 할 Web URL을 클립보드에 복사하는 버튼을 클릭하여 복사해 옴



1. Code 버튼 클릭

2. Web URL 클립보드 복사

- 구글 Colab 셀에 !git clone [클립보드에 복사해온 URL을 Ctrl+V로 붙여넣기] 하면 됨
- 그럼 GitHub로부터 dataset을 다운로드 함

```
# Colab에서 GitHub 에 있는 데이터 가져오기
# haarcascade xml 다운로드 URL : https://github.com/opencv/opencv
# Clone Web URL 를 복사해 옵니다.

!git clone https://github.com/opencv/opencv.git

Cloning into 'opencv'...
remote: Enumerating objects: 309822, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 309822 (delta 0), reused 2 (delta 0), pack-reused 309818
Receiving objects: 100% (309822/309822), 497.50 MiB | 17.10 MiB/s, done.
Resolving deltas: 100% (215711/215711), done.
Checking out files: 100% (7231/7231), done.
```

- VM에 opencv라는 폴더가 생성되었을 것
- 여기서 우리가 필요한 opencv/data/haarcascades만 나의 구글 드라이브로 복사
- haarcascade xml 파일만 복사

- VM에 opencv 폴더가 생성되었는지 확인

```
# 구글 드라이브에 opencv 폴더 확인
```

```
!!s
```

```
gdrive opencv sample_data
```

- 위 코드와 같이 VM에 opencv 폴더가 보일 것임

🔄 opencv/data/haarcascades 폴더를 내 구글 드라이브로 복사 후 opencv 폴더를 제거함

▶ # haarcascade xml 파일만 복사

```
!cp -r opencv/data/haarcascades gdrive/My Drive/CV/Face Detection/data/  
print('folder copy complete!!')
```

📁 folder copy complete!!

opencv/data/haarcascades 폴더를
내 구글 드라이브로 복사하기

▶ # 다운로드 받았던 파일 제거(선택사항)

```
!rm -r opencv
```

[] # 구글 드라이브에 opencv 폴더 제거 확인

```
!!ls
```

```
gdrive sample_data
```

VM의 opencv 폴더 제거하기

- 필요한 패키지와 모듈을 불러옴
- 구글 Colab 노트북 화면에서 실행 버튼 클릭



필요한 패키지와 모듈을 불러온다.

```
import cv2  
from google.colab.patches import cv2_imshow  
import numpy as np
```

- OpenCV 버전을 확인
- 구글 Colab 노트북 화면에서 실행 버튼을 클릭



OpenCV 버전 확인

```
print("OpenCV version:")  
print(cv2.__version__)
```



```
OpenCV version:  
4.6.0
```

- ⌚ 얼굴 검출하기 전에 원본 이미지를 화면에 출력
- ⌚ 얼굴 정면이 보이는 이미지 사진을 한 장 읽어와서 img1 객체에 저장

```
1 # 얼굴 검출하기 전에 원본 이미지를 Display 한다.  
2  
3 img1 = cv2.imread("gdrive/My Drive/CV/Face Detection/data/image/face_12.jpg")  
4 print("shape: {}".format(img1.shape))          # shape: (683, 1024, 3)  
5 print("width: {} pixels".format(img1.shape[1]))  
6 print("height: {} pixels".format(img1.shape[0]))  
7 print("channels: {}".format(img1.shape[2]))  
8  
9 (height, width) = img1.shape[:2]  
10 cv2.imshow(img1)                                # 이미지 출력
```



- ⌚ 얼굴 검출하기 전에 원본 이미지를 화면에 출력
- ⌚ 얼굴 정면이 보이지 않는 이미지 사진을 한 장 읽어와서 img2 객체에 저장

```
1 # 얼굴 검출하기 전에 원본 이미지를 Display 한다.  
2  
3 img2 = cv2.imread("gdrive/My Drive/CV/Face Detection/data/image/face_05.jpg")  
4 print("shape: {}".format(img2.shape))          # shape: (854, 1280, 3)  
5 print("width: {} pixels".format(img2.shape[1]))  
6 print("height: {} pixels".format(img2.shape[0]))  
7 print("channels: {}".format(img2.shape[2]))  
8  
9 (height, width) = img2.shape[:2]  
10 cv2.imshow(img2)                                # 이미지 출력
```



- ⌚ 얼굴 검출하기 전에 원본 이미지를 화면에 출력
- ⌚ 배구 선수 이미지 사진을 한 장 읽어와서 img3 객체에 저장

```
1 # 얼굴 검출하기 전에 원본 이미지를 Display 한다.  
2  
3 img3 = cv2.imread("gdrive/My Drive/CV/Face Detection/data/image/sports_02.jpg")  
4 print("shape: {}".format(img3.shape))          # shape: (333, 500, 3)  
5 print("width: {} pixels".format(img3.shape[1]))  
6 print("height: {} pixels".format(img3.shape[0]))  
7 print("channels: {}".format(img3.shape[2]))  
8  
9 (height, width) = img3.shape[:2]  
10 cv2.imshow(img3)                                # 이미지 출력
```



- ⌚ 얼굴 검출하기 전에 원본 이미지를 화면에 출력
- ⌚ 축구 선수 이미지 사진을 한 장 읽어와서 img4 객체에 저장

```
1 # 얼굴 검출하기 전에 원본 이미지를 Display 한다.  
2  
3 img4 = cv2.imread("gdrive/My Drive/CV/Face Detection/data/image/sports_01.jpg")  
4 print("shape: {}".format(img4.shape))          # shape: (349, 620, 3)  
5 print("width: {} pixels".format(img4.shape[1]))  
6 print("height: {} pixels".format(img4.shape[0]))  
7 print("channels: {}".format(img4.shape[2]))  
8  
9 (height, width) = img4.shape[:2]  
10 cv2.imshow(img4)                                # 이미지 출력
```



- ⌚ 차량 번호판 인식하기 전에 원본 이미지를 화면에 출력
- ⌚ 주차된 자동차 이미지 사진을 한 장 읽어와서 img5 객체에 저장

```
1 # 차량번호 검출하기 전에 원본 이미지를 Display 한다.  
2  
3 img5 = cv2.imread("gdrive/My Drive/CV/Face Detection/data/image/car2.jpg")  
4 print("shape: {}".format(img5.shape))          # shape: (721, 960, 3)  
5 print("width: {} pixels".format(img5.shape[1]))  
6 print("height: {} pixels".format(img5.shape[0]))  
7 print("channels: {}".format(img5.shape[2]))  
8  
9 (height, width) = img5.shape[:2]  
10 cv2_imshow(img5)                                # 이미지 출력
```



- 특정값이 미리 정의되어 있는 haarcascade module(xml 파일) 위치를 정의함
- cv2.CascadeClassifier() 함수를 통해 정면 얼굴, 측면 얼굴, 눈, 상반신, 웃음, 자동차 번호판 인스턴스를 생성

haarcascade_frontalface_alt.xml	전면 얼굴 기본 검출기 xml 파일
haarcascade_profileface.xml	측면 얼굴을 인식하는 기본 검출기 xml 파일
haarcascade_eye_tree_eyeglasses.xml	눈을 인식하는 기본 검출기 xml 파일
haarcascade_upperbody.xml	상반신을 인식하는 기본 검출기 xml 파일
haarcascade_smile.xml	웃음을 인식하는 기본 검출기 xml 파일
haarcascade_Russian_plate_number.xml	자동차 번호판을 인식하는 기본 검출기 xml 파일


```
1 # 특성값이 미리 정의되어 있는 haarcascade feature 파일 xml 위치를 정의
2
3 # 정면 얼굴을 인식하는 feature 파일
4 front_face_cascade_name = 'gdrive/My Drive/CV/Face Detection/data/haarcascades/haarcascade_frontalface_alt.xml'
5
6 # 측면 얼굴을 인식하는 feature 파일
7 side_face_cascade_name = 'gdrive/My Drive/CV/Face Detection/data/haarcascades/haarcascade_profileface.xml'
8
9 # 눈을 인식하는 feature 파일
10 eyes_cascade_name = 'gdrive/My Drive/CV/Face Detection/data/haarcascades/haarcascade_eye_tree_eyeglasses.xml'
11
12 # 사람의 상반신을 인식하는 feature 파일
13 upper_body_cascade_name = 'gdrive/My Drive/CV/Face Detection/data/haarcascades/haarcascade_upperbody.xml'
14
15 # 사람의 웃음 인식하는 feature 파일
16 smile_cascade_name = 'gdrive/My Drive/CV/Face Detection/data/haarcascades/haarcascade_smile.xml'
17
18 # 자동차 번호판을 인식하는 feature 파일
19 plate_number_cascade_name = 'gdrive/My Drive/CV/Face Detection/data/haarcascades/haarcascade_russian_plate_number.xml'
20
21 # front_face_cascade 인스턴스를 생성한다.
22 front_face_cascade = cv2.CascadeClassifier()
23
24 # side_face_cascade 인스턴스를 생성한다.
25 side_face_cascade = cv2.CascadeClassifier()
26
27 # eyes_cascade 인스턴스를 생성한다.
28 eyes_cascade = cv2.CascadeClassifier()
29
30 # upperbody_cascade 인스턴스를 생성한다.
31 upperbody_cascade = cv2.CascadeClassifier()
32
33 # smile_cascade 인스턴스를 생성한다.
34 smile_cascade = cv2.CascadeClassifier()
35
36 # plate_number_cascade 인스턴스를 생성한다.
37 plate_number_cascade = cv2.CascadeClassifier()
```

- load() 함수로 Haar Cascade 특성값이 미리 정의되어 있는 xml 파일을 불러옴
- 여기서 이미지 width size = 650으로 지정

```
1 # cascades 파일을 불러온다.
2
3 if not front_face_cascade.load(cv2.samples.findFile(front_face_cascade_name)): # 사람의 정면 얼굴
4     print('--(!)Error loading face cascade')
5     exit(0)
6
7 if not side_face_cascade.load(cv2.samples.findFile(side_face_cascade_name)): # 사람의 측면 얼굴
8     print('--(!)Error loading face cascade')
9     exit(0)
10
11 if not eyes_cascade.load(cv2.samples.findFile(eyes_cascade_name)): # 사람의 눈
12     print('--(!)Error loading eyes cascade')
13     exit(0)
14
15 if not upperbody_cascade.load(cv2.samples.findFile(upper_body_cascade_name)): # 사람의 상반신
16     print('--(!)Error loading eyes cascade')
17     exit(0)
18
19 if not smile_cascade.load(cv2.samples.findFile(smile_cascade_name)): # 웃음
20     print('--(!)Error loading eyes cascade')
21     exit(0)
22
23 if not plate_number_cascade.load(cv2.samples.findFile(plate_number_cascade_name)): # 자동차 번호판
24     print('--(!)Error loading eyes cascade')
25     exit(0)
26
27 # 이미지 width size 지정
28 frame_width = 650
```


- frame_width(여기서는 650) 사이즈에 맞추어 이미지 크기를 재조정하는 imageReSize() 함수를 정의함

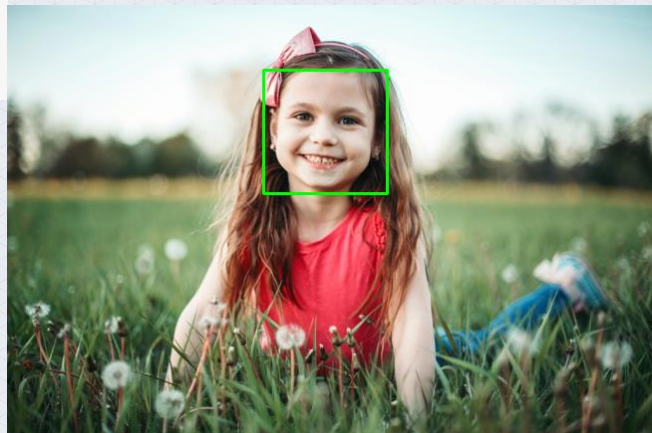
여기서는 컬러(3채널) 이미지를 반환함
(흑백(1채널) 이미지를 반환하는 경우 채널을 BGR to GRAY로 변환)

```
1 # frame_width 에 맞춰 image resize
2
3 def imageReSize(image):
4     (height, width) = image.shape[:2]
5     ratio = frame_width / width
6     dimension = (frame_width, int(height * ratio))
7     image = cv2.resize(image, dimension, interpolation = cv2.INTER_AREA)
8     # image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY) # 채널이 많으면 인식률이 떨어짐으로 하나의 채널로 변환(BGR to GRAY)
9     # image = cv2.equalizeHist(image) # 디지털 이미지처럼 단순화함
10    return image
```

■ img1 이미지를 이용해 사람의 정면 얼굴을 인식해보자

🔄 그림과 같이 정면 얼굴이 잘 인식된 것을 볼 수 있음

```
1 # img1 사람의 정면 얼굴 인식
2
3 image_resized = imageReSize(img1)
4 face_detections = front_face_cascade.detectMultiScale(image_resized)
5
6 for (x, y, w, h) in face_detections:
7     cv2.rectangle(image_resized, (x, y), (x + w, y + h), (0, 255, 0), 2)
8
9 cv2.imshow(image_resized)
```



■ img1 이미지를 이용해 사람의 웃음을 인식해보자

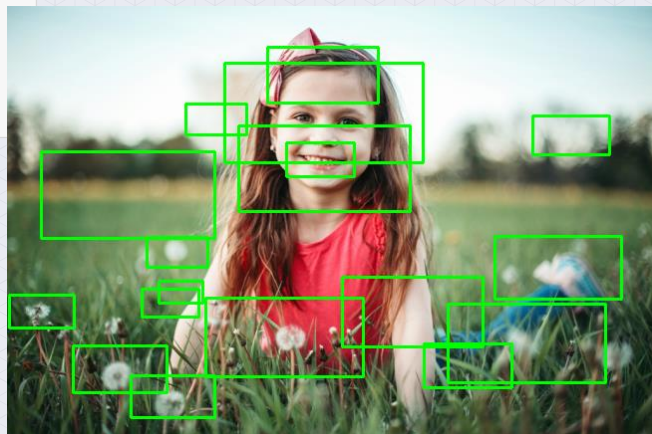
- detectMultiScale() 함수의 scaleFactor=1.1, minNeighbors=4로 설정
- 그림과 같이 웃음이 18개로 잘못 인식됨

```
# img1 웃음 인식

image_resized = imageReSize(img1)
face_detections = smile_cascade.detectMultiScale(image_resized, scaleFactor=1.1, minNeighbors=4)
print(face_detections.shape)      # (18, 4)

for (x, y, w, h) in face_detections:
    cv2.rectangle(image_resized, (x, y), (x + w, y + h), (0, 255, 0), 2)

cv2.imshow(image_resized)
```



■ img1 이미지를 이용해 사람의 웃음을 인식해보자

- ➡ 그림과 같이 웃음이 잘 인식됨
- ➡ detectMultiScale() 함수의 scaleFactor=2.5, minNeighbors=6로 설정

```
# img1 웃음 인식

image_resized = imageResized(img1)
face_detections = smile_cascade.detectMultiScale(image_resized, scaleFactor=2.5, minNeighbors=6)
print(face_detections.shape)      # scaleFactor=1.1 (19, 4) / scaleFactor=2.5 (6.4)

for (x, y, w, h) in face_detections:
    cv2.rectangle(image_resized, (x, y), (x + w, y + h), (0, 255, 0), 2)

cv2.imshow('image_resized')
```



참고

`Cascade.detectMultiScale(image, scaleFactor, minNeighbors)``Cascade.detectMultiScale(image, scaleFactor=1.1, minNeighbors=4)`

■ scaleFactor 파라미터

- ↻ scaleFactor 파라미터는 이미지 스케일과 관련 있는 파라미터임
- ↻ 이미지에서 먼 거리에 있는 얼굴은 작게 보이고, 가까이 있는 얼굴은 크게 보임
그래서 작은 얼굴은 감지하기가 어려움
- ↻ 이런 경우 scaleFactor 파라미터를 조정해서 크기를 조정하며 얼굴을 감지할 수 있음
- ↻ 큰 얼굴은 작게, 작은 얼굴을 크게 조정하면서 감지하는 것
- ↻ scaleFactor의 기본값은 1.1인데, 이보다 작으면 더 많은 검출을 함
- ↻ scaleFactor가 클수록 검출하는 개수가 적어짐
- ↻ 가령, scaleFactor=10으로 크게 설정하면 아무런 얼굴도 검출하지 못할 것임

참고

`Cascade.detectMultiScale(image, scaleFactor, minNeighbors)``Cascade.detectMultiScale(image, scaleFactor=1.1, minNeighbors=4)`

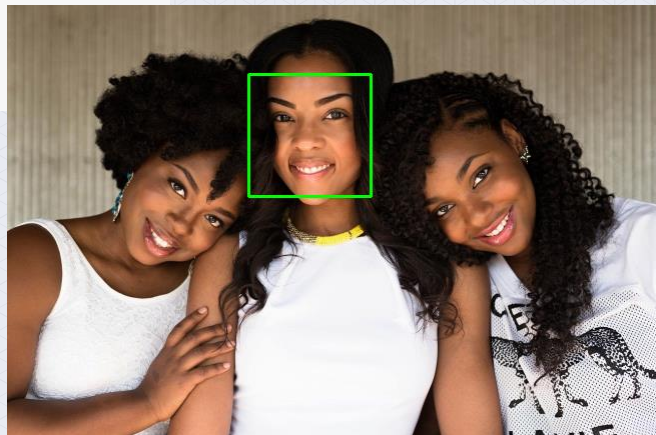
■ minNeighbors 파라미터

- ⌚ 캐스케이드 검출기로 검출을 하면 얼굴 주변에 여러 개의 후보 경계 박스(candidate bounding boxes)를 생성함
- ⌚ 여러 후보 경계 박스 가운데 가장 얼굴을 잘 둘러싸는 경계 박스를 최종적으로 선택함
- ⌚ minNeighbors 파라미터는 최종 경계 박스를 선택하기 위해 얼굴 주변에 존재해야 하는 최소 후보 경계 박스 개수임
- ⌚ 만약에 minNeighbors=5라면 한 얼굴에 최소한 5개의 후보 경계 박스가 있어야 해당 얼굴을 검출함

■ img2 이미지를 이용해 사람의 정면 얼굴을 인식해보자

- 🔄 그림과 같이 한 사람의 정면 얼굴이 잘 인식됨
- 🔄 detectMultiScale() 함수의 scaleFactor=1.1, minNeighbors=4로 설정

```
1 # img2 사람의 정면 얼굴 인식
2
3 image_resized = imageResized(img2)
4 face_detections = front_face_cascade.detectMultiScale(image_resized, scaleFactor=1.1, minNeighbors=4)
5
6 for (x, y, w, h) in face_detections:
7     cv2.rectangle(image_resized, (x, y), (x + w, y + h), (0, 255, 0), 2)
8
9 cv2.imshow('image_resized')
```



■ img2 이미지를 이용해 사람의 측면 얼굴을 인식해보자

🔄 그림과 같이 측면 얼굴로 인식된 것이 없음

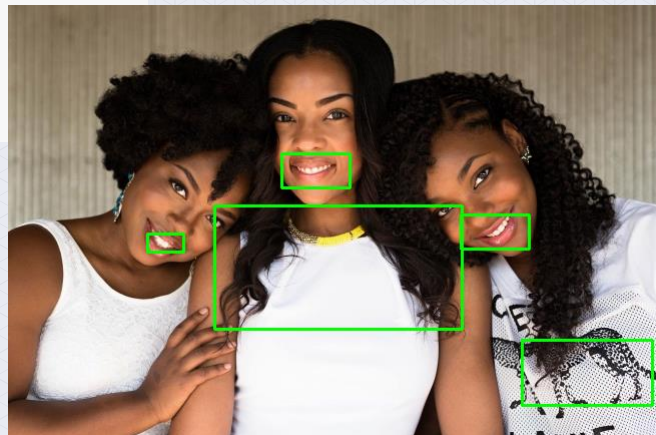
```
1 # img2 사람의 측면 얼굴 인식
2
3 image_resized = imageReSize(img2)
4 face_detections = side_face_cascade.detectMultiScale(image_resized)
5
6 for (x, y, w, h) in face_detections:
7     cv2.rectangle(image_resized, (x, y), (x + w, y + h), (0, 255, 0), 2)
8
9 cv2.imshow(image_resized)
```



■ img2 이미지를 이용해 사람의 웃음을 인식해보자

- ⌚ 그림과 같이 웃음이 5개로 잘못 인식됨
- ⌚ detectMultiScale() 함수의 scaleFactor=1.90, minNeighbors=4로 설정

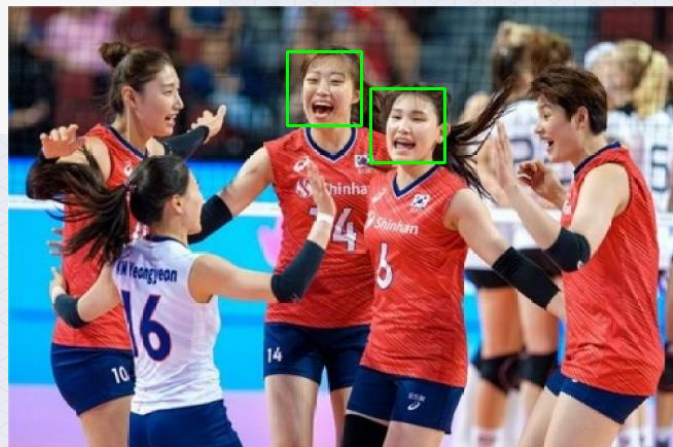
```
1 # img2 웃음 인식
2
3 image_resized = imageResized(img2)
4 face_detections = smile_cascade.detectMultiScale(image_resized, scaleFactor=1.90, minNeighbors=4)
5 print(face_detections.shape)      # (5, 4)
6
7 for (x, y, w, h) in face_detections:
8     cv2.rectangle(image_resized, (x, y), (x + w, y + h), (0, 255, 0), 2)
9
10 cv2.imshow('image_resized')
```



■ img3 이미지를 이용해 사람의 정면 얼굴을 인식해보자

- 🕒 그림과 같이 두 사람의 정면 얼굴이 잘 인식됨
- 🕒 detectMultiScale() 함수의 scaleFactor=1.9, minNeighbors=4로 설정

```
1 # img3 사람의 정면 얼굴 인식
2
3 image_resized = imageResized(img3)
4 face_detections = front_face_cascade.detectMultiScale(image_resized, scaleFactor=1.9, minNeighbors=4)
5
6 for (x, y, w, h) in face_detections:
7     cv2.rectangle(image_resized, (x, y), (x + w, y + h), (0, 255, 0), 2)
8
9 cv2.imshow('image_resized')
```



■ img3 이미지를 이용해 사람의 측면 얼굴을 인식해보자

- ⌚ 그림과 같이 측면 얼굴을 잘 인식하지 못함
- ⌚ detectMultiScale() 함수의 scaleFactor=2.0, minNeighbors=4로 설정

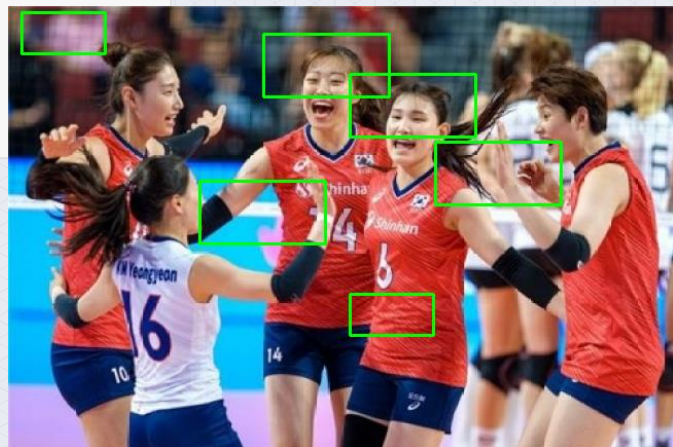
```
1 # img3 사람의 측면 얼굴 인식
2
3 image_resized = imageResize(img3)
4 face_detections = side_face_cascade.detectMultiScale(image_resized, scaleFactor=2.0, minNeighbors=4)
5
6 for (x, y, w, h) in face_detections:
7     cv2.rectangle(image_resized, (x, y), (x + w, y + h), (0, 255, 0), 2)
8
9 cv2.imshow(image_resized)
```



■ img3 이미지를 이용해 사람의 웃음을 인식해보자

- ⌚ 그림과 같이 웃음이 6개로 잘못 인식됨
- ⌚ detectMultiScale() 함수의 scaleFactor=1.5, minNeighbors=7로 설정

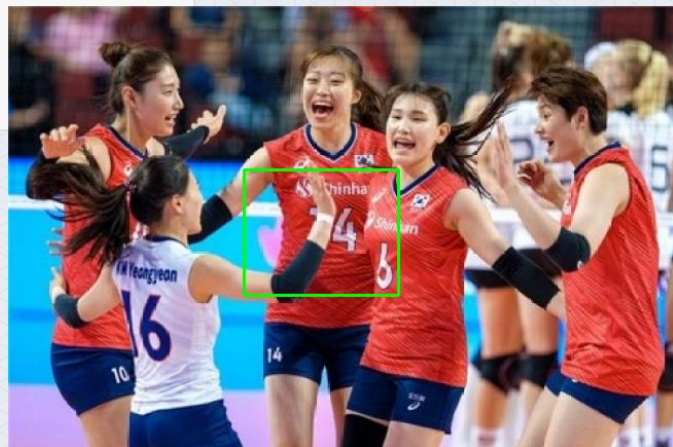
```
1 # img3 웃음 인식
2
3 image_resized = imageResized(img3)
4 face_detections = smile_cascade.detectMultiScale(image_resized, scaleFactor=1.5, minNeighbors=7)
5 print(face_detections.shape)      # (6, 4)
6
7 for (x, y, w, h) in face_detections:
8     cv2.rectangle(image_resized, (x, y), (x + w, y + h), (0, 255, 0), 2)
9
10 cv2.imshow('image_resized')
```



■ img3 이미지를 이용해 사람의 상반신을 인식해보자

- ⌚ 그림과 같이 상반신이 1개로 잘못 인식됨
- ⌚ detectMultiScale() 함수의 scaleFactor=1.2, minNeighbors=4로 설정

```
1 # img3 사람의 상반신 인식
2
3 image_resized = imageResize(img3)
4 face_detections = upperbody_cascade.detectMultiScale(image_resized, scaleFactor=1.2, minNeighbors=4)
5
6 for (x, y, w, h) in face_detections:
7     cv2.rectangle(image_resized, (x, y), (x + w, y + h), (0, 255, 0), 2)
8
9 cv2.imshow('image_resized')
```



■ img4 이미지를 이용해 사람의 정면 얼굴을 인식해보자

- 🔄 그림과 같이 정면 얼굴이 잘 인식됨
- 🔄 detectMultiScale() 함수의 scaleFactor=1.1, minNeighbors=4로 설정

```
1 # img4 사람의 정면 얼굴 인식
2
3 image_resized = imageReSize(img4)
4 face_detections = front_face_cascade.detectMultiScale(image_resized, scaleFactor=1.1, minNeighbors=4)
5
6 for (x, y, w, h) in face_detections:
7     cv2.rectangle(image_resized, (x, y), (x + w, y + h), (0, 255, 0), 2)
8
9 cv2.imshow('image_resized')
```



■ img4 이미지를 이용해 사람의 상반신을 인식해보자

- ⌚ 그림과 같이 상반신이 잘 인식되지 못함
- ⌚ detectMultiScale() 함수의 scaleFactor=1.1, minNeighbors=4로 설정

```
1 # img4 사람의 상반신 인식
2
3 image_resized = imageReSize(img4)
4 face_detections = upperbody_cascade.detectMultiScale(image_resized, scaleFactor=1.1, minNeighbors=4)
5
6 for (x, y, w, h) in face_detections:
7     cv2.rectangle(image_resized, (x, y), (x + w, y + h), (0, 255, 0), 2)
8
9 cv2.imshow('image_resized')
```



■ 다음은 img5 이미지를 이용해 자동차 차량번호판을 인식해보자

- ⌚ 그림과 같이 차량번호판이 잘 인식되지 못함
- ⌚ detectMultiScale() 함수의 scaleFactor=1.1, minNeighbors=4로 설정

```
1 # img5 차량번호 인식
2
3 image_resized = imageResized(img5)
4 face_detections = plate_number_cascade.detectMultiScale(image_resized, scaleFactor=1.1, minNeighbors=4)
5
6 for (x, y, w, h) in face_detections:
7     cv2.rectangle(image_resized, (x, y), (x + w, y + h), (0, 255, 0), 2)
8
9 cv2.imshow('image_resized')
```



detectAndDisplay() 함수는 이미지에서 정면 얼굴, 눈, 웃음을 인식하고 화면 출력하는 함수를 정의

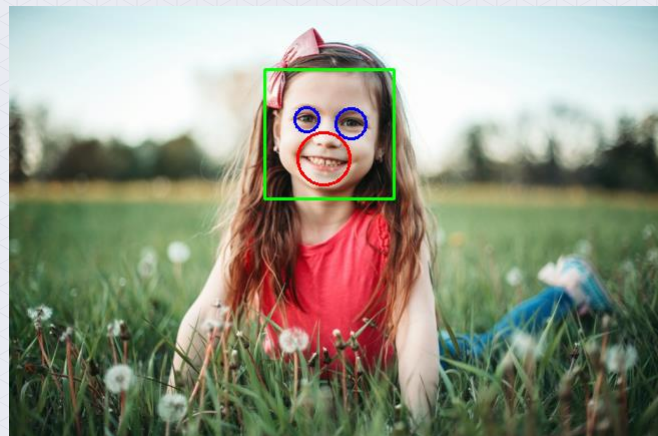
```

1 # detectAndDisplay() 함수는 Face detection하고 Display하는 함수이다.
2 # - 정면 얼굴 검출, 검출된 얼굴에서 눈과 웃음 인식
3
4 def detectAndDisplay(frame):
5     # frame_width 에 맞춰 image resize
6     (height, width) = frame.shape[:2]
7     ratio = frame_width / width
8     dimension = (frame_width, int(height * ratio))
9     frame = cv2.resize(frame, dimension, interpolation = cv2.INTER_AREA)
10    frame_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY) # 채널이 많으면 인식율이 떨어짐으로 하나의 채널로 변환(BGR to GRAY)
11    frame_gray = cv2.equalizeHist(frame_gray) # 디지털 이미지처럼 단순화함
12
13    # 정면 얼굴을 인식한다.
14    faces = front_face_cascade.detectMultiScale(frame_gray, scaleFactor=1.1, minNeighbors=4)
15    for (x,y,w,h) in faces:
16        center = (x + w//2, y + h//2)
17        frame = cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2) # detection 된 얼굴에 사각형을 그린다.
18        faceROI = frame_gray[y:y+h,x:x+w] # frame_gray에서 발급 그린 얼굴 사각형 영역만 가져온다.(faceROI)
19
20        # faceROI 에서 eyes 을 detection 한다.
21        eyes = eyes_cascade.detectMultiScale(faceROI, scaleFactor=1.1, minNeighbors=4)
22        for (x2,y2,w2,h2) in eyes:
23            eye_center = (x + x2 + w2//2, y + y2 + h2//2)
24            radius = int(round((w2 + h2)*0.25)) # 반지름 계산
25            frame = cv2.circle(frame, eye_center, radius, (255, 0, 0), 2) # detection 된 눈에 원을 그린다.
26
27        # faceROI 에서 웃음을 detection 한다.
28        smiles = smile_cascade.detectMultiScale(faceROI, scaleFactor=1.2, minNeighbors=6)
29        for (x3,y3,w3,h3) in smiles:
30            smile_center = (x + x3 + w3//2, y + y3 + h3//2)
31            radius = int(round((w3 + h3)*0.25)) # 반지름 계산
32            frame = cv2.circle(frame, smile_center, radius, (0, 0, 255), 2) # detection 된 눈에 원을 그린다.
33
34    print("#### Face detection(harcascade) ####")
35
36    # 이미지를 Display 한다.
37    cv2.imshow('frame')
    
```


■ img1 이미지를 이용해 detectAndDisplay() 함수로 사람을 인식해보자

🔄 그림과 같이 정면 얼굴, 눈, 웃음이 잘 인식됨

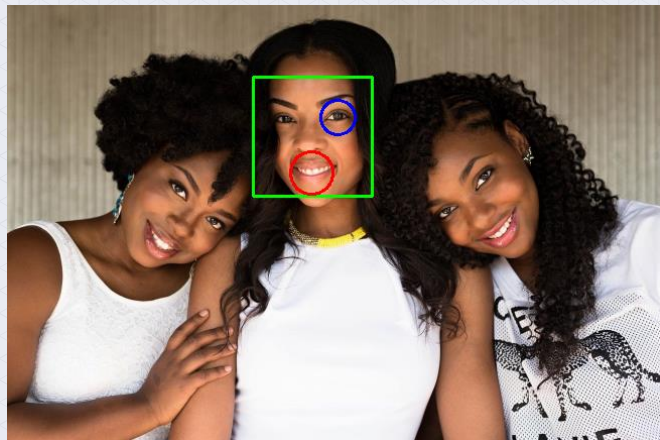
```
1 # img 파일에서 face detection 하고 display 하는 함수 호출
2 detectAndDisplay(img1)
3
4 #
5 # 실행 결과 사람의 정면 얼굴과 눈, 웃음 인식을 모두 잘 하였지만, 눈은 두번 인식한 것을 볼 수 있다.
6 #
```



■ img2 이미지를 이용해 detectAndDisplay() 함수로 사람을 인식해보자

🔄 그림과 같이 정면 얼굴, 눈(한쪽 눈만 인식), 웃음이 잘 인식됨

```
1 # img 파일에서 face detection 하고 display 하는 함수 호출
2 detectAndDisplay(img2)
3
4 #
5 # 실행 결과 정면 얼굴, 눈(한쪽 눈만 인식), 웃음이 잘 인식된 것을 볼 수 있다.
6 #
```



haarcascade module face detection의 특성

／
얼굴 검출 속도는 빠르나 정확도가 떨어짐

／
얼굴이 정면이 아닌 돌려져 있으면 인식률이 떨어짐

／
이미지의 크기(해상도)가 작아지면 인식률도 떨어짐

／
서양인에 비해 동양인 인식률이 떨어짐